

Offline Signature Forgery Detection with EfficientNet-B0 and Batch-Hard Triplet Loss Fine-Tuning

1st David Nehemia Sunoto

Department / Faculty

University Name

City, Country

NIM: 2902726965

2nd Febrianto

Department / Faculty

University Name

City, Country

NIM: 2902727015

3rd Frolentika

Department / Faculty

University Name

City, Country

NIM: 2902727053

4th Jessica Davalynda

Department / Faculty

University Name

City, Country

NIM: 2902727091

5th Windy Sulistiawati

Department / Faculty

University Name

City, Country

NIM: 2902727280

Abstract—Handwritten signatures are still one of the most widely accepted ways to authenticate financial, legal, and administrative documents. Checking them by hand, however, is slow, subjective, and often unreliable once a forgery has been practiced. This paper presents a writer-independent offline signature verification system built around a deliberately simple idea: rather than retraining a deep network end to end, we freeze a pretrained EfficientNet-B0 backbone and fine-tune only its embedding head with batch-hard triplet loss. The head was trained on genuine signatures from 221 writers pooled from CEDAR, GPDS, and a locally collected dataset. Each signature is mapped to a 512-dimensional L2-normalized embedding, and pairs are compared by cosine similarity. Evaluation used 33 writers held out entirely from training, and we report random-impostor and skilled-forgery performance separately, a distinction that much of the literature omits. Fine-tuning raised the random-impostor Area Under the Curve (AUC) from 0.953 to 0.979, with the Equal Error Rate falling from 10.71% to 7.07%, and raised the skilled-forgery AUC from 0.817 to 0.871, with the Equal Error Rate falling from 25.25% to 20.20%. Verification accuracy reached 90.20% against random impostors and 79.49% against skilled forgeries. The whole procedure runs in minutes on a CPU, and every reported number comes from a single reproducible evaluation script, making the result a practical baseline for cross-dataset offline signature verification.

Keywords—EfficientNet, Metric Learning, Offline Signature Verification, Skilled Forgery Detection, Triplet Loss

I. INTRODUCTION

Handwritten signatures remain a legally recognized and socially trusted way of authorizing banking transactions, contracts, and official documents, even though digital alternatives have existed for years. In day-to-day practice, authenticity is still mostly judged by a person comparing a questioned signature against known references. That process does not scale well. It is subjective, tiring, and prone to disagreement between examiners, and it becomes genuinely unreliable when the questioned sample is a *skilled forgery* that

someone has practiced until its shape, slant, and stroke rhythm look right [1]. These weaknesses have driven decades of work on automated offline signature verification (OSV), in which a system must decide from a static scanned image alone whether a signature really belongs to the claimed writer [1].

Approaches to OSV have changed considerably over time. Early systems paired handcrafted descriptors, such as the Scale-Invariant Feature Transform (SIFT) and the Histogram of Oriented Gradients (HOG), with Support Vector Machine (SVM) classifiers. Those pipelines were easy to interpret, but they generalized poorly across writers because the notion of similarity was designed by hand instead of learned from data. Deep Convolutional Neural Networks (CNNs) changed this picture, and compound-scaled architectures such as EfficientNet showed that high accuracy does not require an enormous model [2]. Transfer learning became the default recipe, where a network pretrained on ImageNet is fine-tuned for the signature task [3], [4]. Metric learning pushed things further: Siamese and triplet-loss formulations, which learn an embedding space rather than a fixed classifier, are now the usual choice for writer-independent verification [5], [6], [7]. Self-supervised Vision Transformers such as DINOv2 [8] mark the current frontier in representation learning, although they have barely been tried on signatures so far.

Even with this progress, a gap remains. Public OSV benchmarks are small. CEDAR contains only 55 writers, and results that appear nearly perfect on one dataset do not automatically carry over to another; an Equal Error Rate (EER) below 0.06% has been reported on CEDAR alone [9], which says little about cross-dataset, writer-independent behavior. The bigger issue is that skilled forgeries are far harder to reject than random impostor signatures, yet many papers publish one aggregate accuracy that blends the two attack types. A system can look excellent on average and still be fooled by anyone patient enough to practice. On top of that, most CNN transfer-learning work fine-tunes the whole network on a single dataset, which is computationally expensive and makes it hard to tell how much of the gain comes from the learned embedding rather than the pretrained backbone.

This study tries to close these gaps with three objectives. The first is to build a writer-independent EfficientNet-B0 embedding model and evaluate it jointly across CEDAR, GPDS, and a locally collected signature dataset. The second is to design and test a lightweight fine-tuning strategy in which the backbone stays frozen and only the embedding head is retrained with batch-hard triplet loss, so that the head's contribution can be measured in isolation. The third is to quantify performance against random-impostor and skilled-forgery attacks separately, on a held-out set whose writers never appear in training. We consider this kind of transparent reporting important, because the skilled-forgery case is the one that matters most in practice and it is also the one most often glossed over in the literature.

II. LITERATURE REVIEW

A. EfficientNet

EfficientNet is a family of convolutional networks introduced by Tan and Le [2]. Its central idea, compound scaling, adjusts network depth, width, and input resolution together through a single coefficient, whereas earlier architectures tended to scale one of these dimensions at a time based on intuition. The smallest member, EfficientNet-B0, was found through neural architecture search. It reaches accuracy comparable to much larger networks such as ResNet-50 and Inception-v3 while using roughly an order of magnitude fewer parameters and floating-point operations. That efficiency matters for biometric problems, where labeled data is scarce and a compact backbone is less likely to overfit a few thousand images. Signature verification pipelines have accordingly adopted EfficientNet variants through transfer learning, reusing ImageNet-pretrained features for genuine-versus-forged discrimination [3], [4], usually by swapping the classification layer for a task-specific head. This study follows the same pattern.

B. Triplet Loss and Metric Learning

A conventional classifier assigns an image to one of a fixed set of classes. Metric learning takes a different route: it learns an embedding function so that the distance between two embeddings reflects whether they share an identity. This fits writer-independent verification naturally, since the writers encountered at deployment are unknown at training time. The triplet loss, popularized for face verification by Schroff et al. [5], trains such an embedding using an anchor, a positive sample of the same identity, and a negative sample of a different one, penalizing cases where the negative is not further from the anchor than the positive by some margin. Naive triplet sampling has a known weakness, though. Once basic separation is achieved, most randomly drawn triplets are already easy and contribute almost no gradient. Hermans et al. [6] proposed batch-hard mining as a remedy: for each anchor in a batch, only the hardest positive and the hardest negative actually present in that batch are used. We adopt that strategy directly. Within the signature domain, Dey et al. introduced SigNet, a Siamese convolutional network trained with a contrastive objective, and showed that learned embeddings beat handcrafted descriptors for writer-independent verification [7].

C. Related Works

Al-Azzani and Musleh (2024) [3] used Particle Swarm Optimization (PSO) to search the hyperparameters of a CNN-based offline signature verifier, tuning filter counts, dense-layer width, dropout, and learning rate. Tested across GPDS, CEDAR, BHSig260-Bengali, and BHSig260-Hindi, the approach reached a testing accuracy of 98.3%. The result is a useful reminder that careful hyperparameter search can rival more elaborate architectural changes. The paper reports a single aggregate accuracy, however, without separating random-impostor from skilled-forgery cases, which makes it difficult to compare against forgery-specific robustness claims.

Chokshi et al. (2023) [9] proposed SigScatNet, a Siamese network combined with Scattering wavelet transforms, aimed at low-resource hardware. On CEDAR the method reached an EER of 0.0578%, and on the ICDAR SigComp Dutch dataset 3.689%. The CEDAR figure amounts to near-perfect separation, but the sixty-fold gap between the two datasets is telling: CEDAR, with its 55 writers and fixed forgery set, appears easy to saturate and cannot by itself demonstrate writer-independent generalization. This observation motivated the multi-dataset protocol used here.

Ozyurt et al. (2024) [4] used MobileNetV2 as a transfer-learning feature extractor on 12,600 signature images from 420 individuals, 30 genuine signatures each, then applied feature selection with Neighborhood Component Analysis (NCA), Chi-squared, and mutual-information criteria before classifying with SVM and k-nearest-neighbor classifiers. Selecting 300 NCA features lifted accuracy from 91.3% to 97.7%, a clear sign that transfer-learned CNN features benefit from refinement rather than being used as-is. Since the dataset contains only genuine signatures, though, the task is closer to writer identification than to forgery detection, and the harder skilled-forgery problem that concerns the present work is left open.

III. METHODOLOGY

This section describes how the verification system was built and evaluated. The pipeline runs from dataset preparation and preprocessing, through frozen-backbone feature extraction and embedding-head fine-tuning, to checkpoint selection and a final held-out evaluation under two attack scenarios. Throughout, the priority was a fair, writer-disjoint evaluation that others can reproduce. Everything was implemented in Python with PyTorch and executed on CPU hardware only; backbone features were computed once and cached to disk, which keeps training practical without a GPU.

A. Dataset

Three signature sources were combined for this study. The CEDAR database contributes 55 writers, each with 24 genuine and 24 skilled-forged signatures. A locally available subset of the GPDS database contributes 150 writers, averaging 8 genuine and 14 skilled-forged signatures per writer. A locally collected dataset adds 16 writers with 257 genuine signatures and no forgeries. Only genuine signatures were used for fine-tuning. This mirrors the usual verification setting, in which a system is enrolled on genuine references and meets forgeries for the first time at test time. Table I summarizes the composition of each source.

TABLE I. DATASET COMPOSITION

Dataset	Writers	Genuine	Forged	Role in this study
CEDAR	55	1,320	1,320	Fine-tuning (genuine) + forgery test
GPDS (subset)	150	1,200	2,100	Fine-tuning (genuine) + forgery test
Locally collected	16	257	0	Fine-tuning (genuine) only
Combined pool	221	2,052[†]	3,420*	188 train / 33 held-out

*Forged images were never used for training; they were reserved exclusively for the skilled-forgery test in Section III-B and the evaluation in Section IV.

[†]After capping at twelve genuine images per writer (660 CEDAR + 1,200 GPDS + 192 local); the full per-source totals sum to 2,777.

To keep heavily sampled writers from dominating the pool, each writer contributed at most twelve genuine images, which yields the working pool of 2,052 images across 221 writers shown in Table I. This pool was then split at the writer level rather than the image level: 188 writers for fine-tuning and 33 writers held out entirely for validation and testing. A fixed random seed makes the split reproducible, and keeping writers disjoint between partitions is what allows a writer-independent claim in the first place. The held-out set ended up containing 18 GPDS writers, 12 CEDAR writers, and 3 locally collected writers.

B. Proposed Method

The proposed method follows the pipeline in Fig. 1: preprocessing and feature extraction first, then embedding-head fine-tuning with batch-hard triplet loss, and finally checkpoint selection and held-out evaluation.

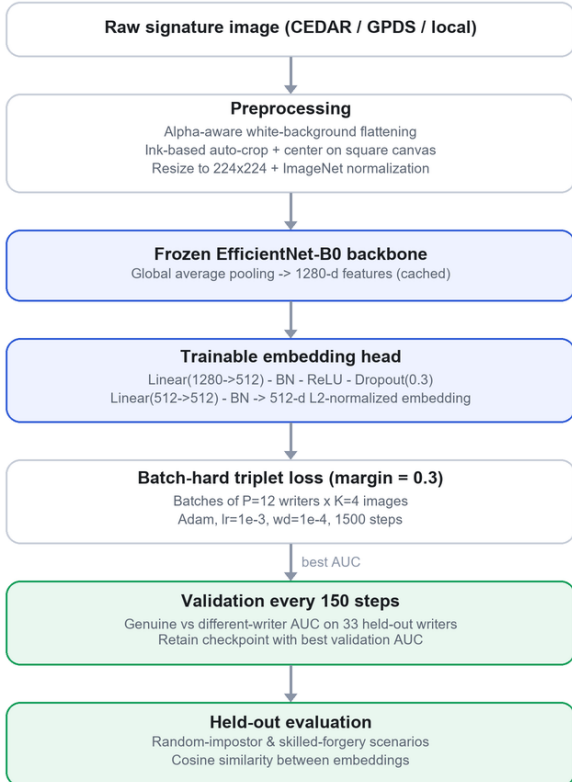


Fig. 1. System flowchart of the proposed pipeline.

Preprocessing starts by bringing every image to a consistent RGB form. Images with genuine alpha-channel transparency are composited onto a white background, while fully opaque images pass through untouched. The distinction matters, because treating an opaque image as if it were transparent flattens the whole frame to black, an artifact we actually encountered during development. The signature is then auto-cropped to its ink bounding box with a small margin, centered on a square canvas, resized to 224×224 pixels, and normalized with the ImageNet channel statistics.

The embedding network is an EfficientNet-B0 backbone followed by a projection head. The backbone weights come from a checkpoint previously trained end-to-end with triplet loss on GPDS. The head consists of a linear layer (1280 to 512), batch normalization, a rectified linear unit, dropout at 0.3, a second linear layer (512 to 512), and a final batch-normalization layer; the output is L2-normalized onto the unit hypersphere. This pretrained model, taken as-is, serves as the baseline against which the proposed fine-tuning is measured.

Since the backbone had already been trained on a related distribution, we expected most of the remaining gain to come from re-adapting the head rather than the low-level convolutional filters. The backbone was therefore frozen completely, and only the head was fine-tuned on the 221-writer genuine-only pool. The 1,280-dimensional backbone features were pre-computed once and cached, which is what makes CPU-only training feasible. Fine-tuning used batch-hard triplet loss with a margin of 0.3 over cosine distance; each batch drew twelve writers at random with four images per writer. We optimized with Adam at a learning rate of 1×10^{-3} and a weight decay of 1×10^{-4} for 1,500 steps, computed validation AUC on the 33 held-out writers every 150 steps, and kept the checkpoint with the best validation AUC as the final model.

IV. RESULTS AND DISCUSSION

This section compares the two stages of the model, the pretrained baseline and the head-fine-tuned version, on the 33 held-out writers. Verification works by embedding each signature into the 512-dimensional space and scoring pairs with cosine similarity. Same-writer genuine pairs form the positive class, 495 pairs in total. Negatives come in two forms: genuine signatures of other writers, which define the random-impostor scenario, and same-writer skilled forgeries, which define the skilled-forgery scenario. Forgeries were available for 30 of the 33 held-out writers.

A. Model Evaluation

Performance is reported with six metrics. The Area Under the ROC Curve (AUC) and the Equal Error Rate (EER), the operating point where false acceptances and false rejections are equally frequent, describe separability independently of any threshold. Accuracy, precision, recall, and F1-score are then computed at one decision threshold per model. That threshold was chosen at the equal-error point over the combined pool of random and skilled negatives, mimicking a deployment in which the system cannot know in advance which attack it faces. To keep the threshold-dependent metrics from being distorted by class imbalance, the negative pool of each scenario was

balanced to the number of genuine pairs. Table II shows the results.

TABLE II. EVALUATION RESULTS ON THE 33 HELD-OUT WRITERS

Model	Scenario	AUC	EER	Acc.	Prec.	Rec.	F1
Baseline	Random	0.953	10.71	87.07	92.38	80.81	86.21
Baseline	Skilled	0.817	25.25	74.55	71.81	80.81	76.05
Proposed	Random	0.979	7.07	90.20	95.02	84.85	89.65
Proposed	Skilled	0.871	20.20	79.49	76.64	84.85	80.54

AUC is a ratio in $[0,1]$; EER, Acc., Prec., Rec., and F1 are percentages. Proposed = backbone-frozen head fine-tuning.

Head fine-tuning improved every metric in both scenarios, as Table II shows. Against random impostors, AUC went from 0.953 to 0.979 and EER dropped from 10.71% to 7.07%, while accuracy rose from 87.07% to 90.20%. The skilled-forgery scenario is harder across the board, but it improved as well: AUC from 0.817 to 0.871, EER from 25.25% down to 20.20%, accuracy from 74.55% to 79.49%, and F1-score from 76.05% to 80.54%. The cost of these gains is small. The projection head holds fewer than a million parameters, and with the backbone frozen and features cached, a full fine-tuning run finishes in minutes on an ordinary CPU. Re-shaping the embedding geometry on a multi-dataset corpus was enough to produce these improvements; no end-to-end retraining was needed.

B. Error Analysis

Table III breaks down the confusion of the fine-tuned model at its single operating threshold. Because the positive pairs and the threshold are the same in both scenarios, the genuine side is fixed: 420 of the 495 genuine pairs were accepted, a recall of 84.85%, and 75 were rejected. All of the difference between the scenarios sits on the impostor side.

TABLE III. CONFUSION BREAKDOWN OF THE PROPOSED MODEL

Scenario	Genuine accepted (TP)	Genuine rejected (FN)	Forgery accepted (FP)	Forgery rejected (TN)
Random	420	75	22	473
Skilled	420	75	128	367

Counts are pair-level decisions at the single operating threshold; genuine acceptance is identical across scenarios by construction.

The contrast is stark. Random impostors slipped past the threshold only 22 times out of 495, but skilled forgeries got through 128 times, close to six times as often, and precision dropped accordingly, from 95.02% to 76.64%. The explanation is partly about the attacks themselves and partly about the backbone. A random impostor differs from the reference signature in global shape, slant, and stroke layout, exactly the kind of cues a convolutional embedding picks up easily, so rejecting one is not difficult. A skilled forger, on the other hand, practices until those global cues match. What still separates a genuine signing from a careful imitation lies in fine pen dynamics, and a backbone optimized for natural-image classification does not fully encode them. Skilled forgery therefore stays the dominant error mode after fine-tuning, with an EER of 20.20% against 7.07% for random impostors, roughly a three-to-one ratio.

V. CONCLUSIONS

We presented a deliberately lightweight approach to offline signature verification: freeze a pretrained EfficientNet-B0 embedder and retrain only its projection head with batch-hard triplet loss on a 221-writer corpus combining CEDAR, GPDS, and a locally collected dataset. On 33 writers held out from

training, the fine-tuned head improved random-impostor detection from an AUC of 0.953 to 0.979 (EER from 10.71% to 7.07%) and skilled-forgery detection from an AUC of 0.817 to 0.871 (EER from 25.25% to 20.20%), with verification accuracy reaching 90.20% and 79.49%, respectively. Beyond the raw numbers, we consider the reporting protocol itself a contribution. Random-impostor and skilled-forgery results are stated separately, the partitions are writer-disjoint, and every figure traces back to one evaluation script, which together make the benchmark straightforward to reproduce and to compare against.

Two limitations should be kept in mind. Training touched only the embedding head, because no GPU was available; fine-tuning the backbone end to end, ideally with a larger and more varied pool of skilled forgeries, is the obvious next step for narrowing the skilled-forgery gap that remains the main source of error. In addition, while the evaluation writers never appeared in training, they come from the same dataset families and served double duty for checkpoint selection and testing. A fully independent test partition and a larger held-out set would make the generalization claim stronger. Looking further ahead, models that capture fine stroke dynamics, whether through attention mechanisms, graph-based stroke representations, or self-supervised Vision Transformers such as DINOv2 [8], seem the most promising route to closing the remaining gap against skilled forgeries.

REFERENCES

- [1] L. G. Hafemann, R. Sabourin, and L. S. Oliveira, "Offline handwritten signature verification — Literature review," in Proc. 7th Int. Conf. Image Process. Theory, Tools Appl. (IPTA), Montreal, QC, Canada, 2017, pp. 1–8.
- [2] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. 36th Int. Conf. Mach. Learn. (ICML), Long Beach, CA, USA, 2019, pp. 6105–6114.
- [3] A. M. O. Al-Azzani and A. M. Q. Musleh, "Enhancing offline signature verification through CNN model optimization with PSO algorithm," Int. J. Intell. Syst. Appl. Eng. (IJISAE), vol. 12, no. 3, 2024.
- [4] F. Ozyurt, J. Majidpour, T. A. Rashid, and C. Koc, "Offline handwriting signature verification: A transfer learning and feature selection approach," arXiv:2401.09467, 2024.
- [5] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Boston, MA, USA, 2015, pp. 815–823.
- [6] A. Hermans, L. Beyer, and B. Leibe, "In defense of the triplet loss for person re-identification," arXiv:1703.07737, 2017.
- [7] S. Dey, A. Dutta, J. I. Toledo, S. K. Ghosh, J. Lladós, and U. Pal, "SigNet: Convolutional Siamese network for writer independent offline signature verification," arXiv:1707.02131, 2017.
- [8] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," arXiv:2304.07193, 2023.
- [9] A. Chokshi, V. Jain, R. Bhope, and S. Dhage, "SigScatNet: A Siamese + scattering based deep learning approach for signature forgery detection and similarity assessment," arXiv:2311.05579, 2023.